

COSC 3P93 Project Step 4

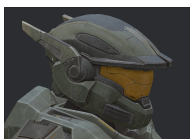
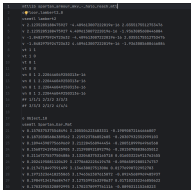
Parker TenBroeck 7376726
pt21zs@brocku.ca



December 5, 2025

Intro

Rendering Triangle Meshes is the process of taking pre-defined or generated meshes and transforming it into a displayable image with a variety of effects



You're probably familiar with this process if you've ever played a 3D game.

Problems

Mesh Representation:

How do we represent a 3D mesh of triangles?

Projection:

How do we take 3D triangles and project them onto our screen?

Rasterization:

How do we draw these triangles to our screen taking into account the correct depths?

Lighting/Texturing:

how do we shade/texture our triangles efficiently?

Pixel Updates:

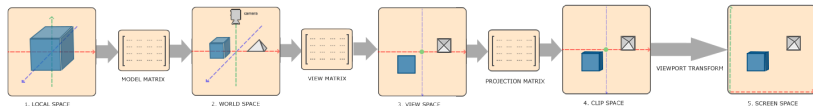
How do we ensure concurrent pixel writes select the closest pixel.

Solutions

Mesh Representation:

Each mesh is a list of Faces, each face containing 3 3D points.

Projection:



Rasterization:

Min/Max box over the triangles projected screen space coordinates. Conversion to barycentric coordinate to calculate linearly interpolated pixel information. Using the projected Z value to determine depth of each pixel.

Lighting/Texturing:

Store needed information in pixel from material. Only calculate lighting on the pixels actually visible (after all triangles have been rasterized). aka "Deferred rendering"

Pixel Updates:

Per Pixel Lock.

Tools and Libraries

STB libraries were used to read/write image files. [1] [2]

Tiny OBJ Loader was used to parse obj files for 3d models. [3]

The 3D models used in the performance tests were sourced from SketchFab. [4] [5]

Learn OpenGL was a incredible resource which all techniques in this project were based off. [6].

Responsibilities

Everything: Me, Myself, And I

Opportunities of Parallelism

Per Frame:

Render separate frames independently of each other.

Per Face(Triangle):

Projection and Rasterization.

Per Fragment(Pixel):

Each Fragment in the image can have lighting and textures calculated independently of each other.

Parallelism Degree

MP:

Each fragment(pixel)'s lighting is calculated truly independently of eachother making this stage embarassingly parallel. Each triangle is projected independently of eachother but the rasterization step needs to ensure atomic stores for pixels in the frame buffer.

MPI:

Embarassingly parallel (for rendering). Each frame is rendered completely independently from eachother making each frame embarassingly parallel.

Parallelism Design

MP:

- ▶ Triangle projection and rasterization is split statically per mesh. Through testing it was found that static scheduling works best in this case.
- ▶ Fragment shader is split statically as all fragments require the same calculations.

MPI: Coordinator/Worker model.

- ▶ Coordinator: is responsible for initiating workers with input parameters (scene, resolution), providing model/image data if requested, and instructing workers which frames to render.
- ▶ Worker: Responsible for requesting needed model/texture data. Rendering requested frames.

MP Implementation

Implementing MP was straight forward only requiring two parallel for clauses. The only synchronization needed was a spin-lock on each pixel to ensure updates happened synchronously

```
static void fragment(ref_mut<FrameBuffer> frame, ref<Scene> scene, ref<ResourceStore> resources) {
    #pragma omp parallel for schedule(static)
    for (usize i = 0; i < frame.size(); i++) {
        frame[i] = frame[i].fragment_shader(scene, resources);
    }
}

static void render_mesh(ref_mut<FrameBuffer> frame, ref<Mesh> mesh, ref<Matrix4<f32>> model_matrix,
    Vector2<f32> screen(static_cast<f32>(frame.width()), static_cast<f32>(frame.height())));
    Matrix3<f32> normal_matrix{ data: model_matrix.inverse().transpose()};
    #pragma omp parallel for schedule(static)
    for (usize i = 0; i < mesh.m_faces.size(); i++) {
        const auto& face = mesh.m_faces[i];

const auto ptr = reinterpret_cast<std::atomic<u32>>*>(&this->depth);
constexpr u32 LOCK_VALUE = 0xFFFFFFFF;
u32 depth;
while ((depth = ptr->exchange(LOCK_VALUE, m: std::memory_order_acquire)) == 0xFFFFFFFF){}
if (depth > pixel.depth) {
    depth = pixel.depth;
    std::memcpy( dest: this, &pixel, m: sizeof(Pixel)-sizeof(pixel.depth));
}
ptr->store(depth, m: std::memory_order_release);
```

MPI Implementation

MPI was slightly more work necessitating workers to request files from the coordinator to load resources. Each worker notifies the coordinator when it is ready to start a new frame, and the coordinator responds with the frame number to render.

```
template<typename T>
void do_receive(T& value, int count, MPI_Datatype datatype) {
    MPI_Status status;
    MPI_Recv(&value, count, datatype, source m_src, static_cast<int>(m_tag), MPI_COMM_WORLD, &status);
    //std::cout << status.MPI_SOURCE << " -> (" << rank() << ") " << " type: " << type_name<T>() <<
}

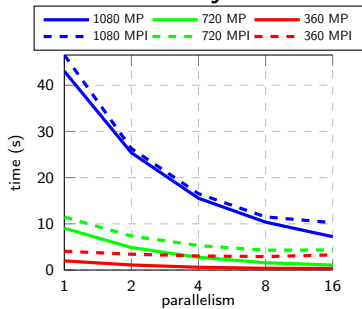
template<typename T>
void do_send(const T& value, int count, MPI_Datatype type) {
    MPI_Send(&value, count, type, m_dest, static_cast<int>(m_tag), MPI_COMM_WORLD);
    // std::cout << "(" << rank() << ") -> " << m_dest << " type: " << type_name<T>()
}
```

Performance Testing

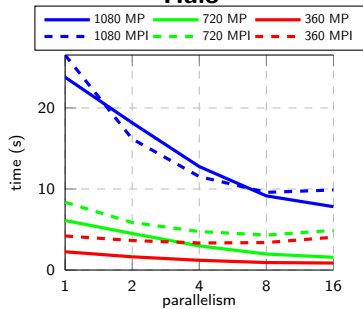
Table: Performance Bench Parameters

Resolution	360x240, 720x480, 1080x1920
Scenes/Triangles	Wavy(7212), Halo(42600), Bricks(752456)
Parallelism	Single Thread, MP/MPI(#1, #2, #4, #8, #16)
Frames	300
CPU	AMD Ryzen 9 7950X (32) @ 5.883GHz
OS	NixOS 25.05 (Warbler) x86_64
RAM	64GiB DDR5
Compiler	clang version 19

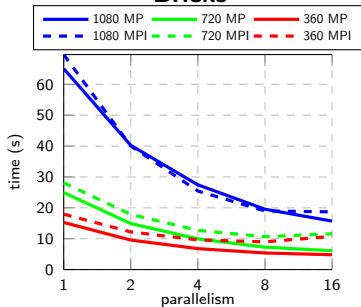
Wavy



Halo



Bricks



MP Performance Analysis

	2	4	8	16
Bricks 360	$S = 1.60\times$ $E = 79.75\%$	$S = 2.25\times$ $E = 56.20\%$	$S = 2.84\times$ $E = 35.46\%$	$S = 3.17\times$ $E = 19.82\%$
Bricks 720	$S = 1.67\times$ $E = 83.69\%$	$S = 2.51\times$ $E = 62.67\%$	$S = 3.44\times$ $E = 42.95\%$	$S = 4.09\times$ $E = 25.54\%$
Bricks 1080	$S = 1.62\times$ $E = 80.83\%$	$S = 2.37\times$ $E = 59.26\%$	$S = 3.33\times$ $E = 41.60\%$	$S = 4.14\times$ $E = 25.90\%$
Halo 360	$S = 1.38\times$ $E = 69.06\%$	$S = 1.87\times$ $E = 46.79\%$	$S = 2.44\times$ $E = 30.48\%$	$S = 2.61\times$ $E = 16.31\%$
Halo 720	$S = 1.36\times$ $E = 67.83\%$	$S = 2.05\times$ $E = 51.26\%$	$S = 3.10\times$ $E = 38.69\%$	$S = 3.89\times$ $E = 24.31\%$
Halo 1080	$S = 1.31\times$ $E = 65.49\%$	$S = 1.86\times$ $E = 46.60\%$	$S = 2.60\times$ $E = 32.50\%$	$S = 3.04\times$ $E = 19.02\%$
Wavy 360	$S = 1.81\times$ $E = 90.32\%$	$S = 3.30\times$ $E = 82.47\%$	$S = 5.28\times$ $E = 65.98\%$	$S = 5.58\times$ $E = 34.89\%$
Wavy 720	$S = 1.86\times$ $E = 93.04\%$	$S = 3.28\times$ $E = 81.89\%$	$S = 5.74\times$ $E = 71.76\%$	$S = 8.38\times$ $E = 52.36\%$
Wavy 1080	$S = 1.70\times$ $E = 84.79\%$	$S = 2.77\times$ $E = 69.24\%$	$S = 4.15\times$ $E = 51.92\%$	$S = 5.96\times$ $E = 37.24\%$
Average	$S = 1.59\times$ $E = 79.42\%$	$S = 2.47\times$ $E = 61.82\%$	$S = 3.66\times$ $E = 45.71\%$	$S = 4.54\times$ $E = 28.38\%$

MPI Performance Analysis

	2	4	8	16
Bricks 360	$S = 1.47\times$ $E = 73.60\%$	$S = 1.87\times$ $E = 46.69\%$	$S = 2.00\times$ $E = 24.96\%$	$S = 1.67\times$ $E = 10.43\%$
Bricks 720	$S = 1.57\times$ $E = 78.69\%$	$S = 2.21\times$ $E = 55.33\%$	$S = 2.64\times$ $E = 33.03\%$	$S = 2.42\times$ $E = 15.15\%$
Bricks 1080	$S = 1.74\times$ $E = 86.98\%$	$S = 2.74\times$ $E = 68.42\%$	$S = 3.67\times$ $E = 45.85\%$	$S = 3.72\times$ $E = 23.27\%$
Halo 360	$S = 1.15\times$ $E = 57.74\%$	$S = 1.26\times$ $E = 31.52\%$	$S = 1.24\times$ $E = 15.50\%$	$S = 1.04\times$ $E = 6.49\%$
Halo 720	$S = 1.43\times$ $E = 71.29\%$	$S = 1.76\times$ $E = 44.02\%$	$S = 1.94\times$ $E = 24.22\%$	$S = 1.71\times$ $E = 10.68\%$
Halo 1080	$S = 1.64\times$ $E = 81.97\%$	$S = 2.30\times$ $E = 57.53\%$	$S = 2.77\times$ $E = 34.60\%$	$S = 2.69\times$ $E = 16.78\%$
Wavy 360	$S = 1.19\times$ $E = 59.32\%$	$S = 1.33\times$ $E = 33.32\%$	$S = 1.40\times$ $E = 17.48\%$	$S = 1.24\times$ $E = 7.73\%$
Wavy 720	$S = 1.56\times$ $E = 77.93\%$	$S = 2.17\times$ $E = 54.17\%$	$S = 2.68\times$ $E = 33.45\%$	$S = 2.63\times$ $E = 16.43\%$
Wavy 1080	$S = 1.77\times$ $E = 88.46\%$	$S = 2.82\times$ $E = 70.41\%$	$S = 4.05\times$ $E = 50.57\%$	$S = 4.54\times$ $E = 28.36\%$
Average	$S = 1.50\times$ $E = 75.11\%$	$S = 2.05\times$ $E = 51.27\%$	$S = 2.49\times$ $E = 31.07\%$	$S = 2.41\times$ $E = 15.03\%$

Showcase

<https://www.cosc.brocku.ca/~pt21zs/demo.html>

Citations



S. Barrett, “stb image.” [Online]. Available: <https://github.com/nothings/stb>



——, “stb image write.” [Online]. Available: <https://github.com/nothings/stb>



S. Fujita, “Tiny obj loader.” [Online]. Available: <https://github.com/tinyobjloader/tinyobjloader>



“Bricks model.” [Online]. Available: <https://sketchfab.com/3d-models/ziegelpfeiler-05ea628cfcea4cbe9837038ef49cd42a>



“Halo model.” [Online]. Available: <https://sketchfab.com/3d-models/spartan-armour-mkv-halo-reach-57070b2fd9ff472c8988e76d8c5cbe66>



“Learn opengl.” [Online]. Available: <https://learnopengl.com/>